

AGENT_ARCHITECTURE

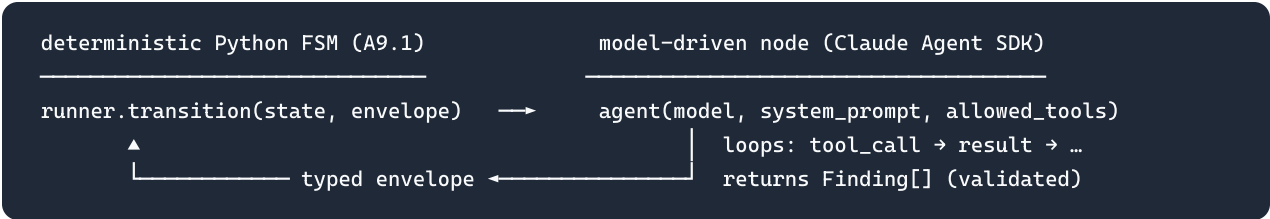
Helios — Agent Architecture & Implementation Spec

Companion to: HELIOS_PROJECT_BIBLE.md §19 (+ §22 memory) · **Version:** v1.0 · **Date:** 2026-06-03 **Specifies artifacts:** A8.0 agent framework / control plane, A8.1 – A8.7 the seven agents, A9.1 pipeline runner, A9.3 audit wiring · **Build milestones:** M7 (minimal loop / L1) → M9 (full loop / L2) (DEPENDENCY_MAP §3). **Reads with:** MCP_ARCHITECTURE.md (tool contracts + the authoritative per-agent allow-list §10) · CLAUDE.md (grounding rules G1–G5, canonical names) · models/semantic/semantic_layer.yaml (the metrics agents may name).

Purpose. Bible §19 is the architecture overview; this doc is what an engineer builds the agent layer against. It pins the control-plane state machine, the typed inter-agent envelope, the seven agents' contracts and system prompts, the Critic refutation loop, the Diagnose hypothesis-tree search, the revenue-at-risk computation, memory integration, the tunable constants, reliability behavior, and the build order.

1. The core design: deterministic control plane, model-driven nodes

Helios is **not** an autonomous free-roaming agent. It is a **deterministic finite state machine** (plain Python — the A9.1 pipeline runner) whose nodes are **Claude agents**. At each node the runner invokes one agent with a fixed system prompt and a fixed MCP tool allow-list; the agent loops tool calls until it returns a **typed JSON envelope**; the runner reads that envelope and decides the next transition. The LLM is a *composer of tool calls*, never the system of record and never the controller.



Consequences: transitions are auditable and reproducible; a node can be unit-tested with recorded tool fixtures; the eval harness can point the same FSM at perturbed data and grade outcomes; and "the agent went rogue and wrote its own SQL" is structurally impossible (no agent holds a raw-SQL tool — MCP doc §10).

2. The seven agents at a glance

(Allowed tools are the authoritative least-privilege set from **MCP_ARCHITECTURE.md §10** — reproduced compactly; that doc is the single source of truth.)

Agent	Model	Responsibility	Consumes	Produces
Orchestrator	Opus	Plan the run, set scope+budget, drive the FSM, route to Critic, finalize	run config, priors	run plan, final brief pointer, <code>run_state</code>
Monitor	Sonnet	Detect which metric/segment series moved abnormally	governed metric series	anomaly list
Decompose	Sonnet	Split each anomaly into mix / rate / interaction	anomaly list	decomposition per anomaly
Diagnose	Opus	Best-first hypothesis-tree RCA, each branch SQL-verified; dollarize leaves	decompositions	candidate <code>Finding[]</code> w/ evidence + revenue-at-risk
Critic	Opus	Adversarially refute every candidate before it ships	<code>Finding</code> + evidence	verdict (PASS/DOWNGRADE/DROP) + reasons
Prescribe	Sonnet	Turn PASS findings into powered experiment cards	PASS <code>Finding[]</code>	hypothesis cards / backlog
Narrator	Sonnet	Compose the exec Decision Brief; persist diagnosis	PASS findings + backlog	brief (md/html), <code>save_diagnosis</code>

2.1 Allowed tools (must match MCP doc §10 exactly)

Agent	Tools
Orchestrator	<code>warehouse.list_tables</code> , <code>warehouse.describe_table</code> , <code>semantic.list_dimensions</code> , <code>report.recall_prior</code>
Monitor	<code>semantic.get_metric</code> , <code>semantic.build_query</code> , <code>warehouse.dry_run</code> , <code>warehouse.run_query</code> , <code>stats.detect_anomaly</code> , <code>stats.forecast</code>
Decompose	<code>semantic.build_query</code> , <code>warehouse.dry_run</code> , <code>warehouse.run_query</code> , <code>stats.decompose_change</code>
Diagnose	<code>semantic.get_metric</code> , <code>semantic.build_query</code> , <code>semantic.list_dimensions</code> , <code>warehouse.dry_run</code> , <code>warehouse.run_query</code> , <code>warehouse.reconcile</code> , <code>stats.significance_test</code> , <code>stats.decompose_change</code> , <code>stats.cohort_retention</code>
Prescribe	<code>experiment.power_analysis</code> , <code>experiment.runtime_estimate</code> , <code>experiment.design_experiment</code> , <code>semantic.get_metric</code>
Critic	<code>semantic.build_query</code> , <code>warehouse.dry_run</code> , <code>warehouse.run_query</code> , <code>warehouse.reconcile</code> , <code>stats.significance_test</code> , <code>stats.decompose_change</code> , <code>report.recall_prior</code>
Narrator	<code>report.render_brief</code> , <code>report.save_diagnosis</code> , <code>report.export</code> , <code>semantic.get_metric</code>

3. Model selection policy

Opus is reserved for the three roles with an open hypothesis space and high cost-of-error: **Orchestrator** (global planning, budget allocation, branch routing), **Diagnose** (combinatorial hypothesis-tree search with self-directed SQL verification), **Critic** (adversarial reasoning that must out-think Diagnose to find a confound). **Sonnet** runs the four bounded-action-space stages — **Monitor, Decompose, Prescribe, Narrator** — where the next move is largely determined (which series to test, which decomposition to run, which power analysis to call, which template to fill). This split keeps token cost inside budget while preserving accuracy on the reasoning-critical paths. Models are config (§9), overridable per deployment.

4. The agent framework (A8.0)

The framework is the shared substrate every agent runs on. It is the only instrumentation and enforcement point.

4.1 Agent configuration shape

```
@dataclass(frozen=True)
class AgentSpec:
    name: str # "Diagnose"
    model: str # "claude-opus-4-8" | "claude-sonnet-4-6"
    system_prompt: str # role + grounding rules + output contract (§6)
    allowed_tools: list[str] # exactly the §2.1 row; enforced, not advisory
    max_tool_calls: int # per-node budget guard
    temperature: float = 0.0 # low; determinism where the model allows
    output_schema: dict # JSON Schema of the envelope the node must return
```

4.2 The tool-call wrapper (enforcement + audit + budget + retries)

Every tool call from every agent passes through one wrapper. This is where the system's invariants live:

```
on tool_call(agent, tool, args):
    assert tool in agent.allowed_tools # least privilege (else AllowListViolati
    if tool == "run_query": assert dry_run_seen(args.sql) # G3 (else NotDryRunFirst)
    enforce_running_byte_budget(args) # tighten scope if near 5 GiB cap
    t0 = clock()
    result = call_with_retries(tool, args) # transient → backoff ≤3 (§8)
    append_audit_log(run_id, step_seq++, agent, tool, # §22.5, A9.3
                    args_hash, sql_text, bytes_scanned, latency=clock()-t0, verdict)
    return result
```

The wrapper guarantees: no agent calls a tool outside its allow-list; `run_query` is always preceded by `dry_run`; every call is on the audit trail (proving "100% governed SQL"); and the per-run byte budget is enforced mid-run.

4.3 The inter-agent envelope — the `Finding`

All hand-offs are typed JSON `Finding` objects (never prose). This is the spine of the pipeline; raw query rows are summarized to aggregates before entering any LLM context (§4.4).

```
Finding = {
    "finding_id": "F-2021-0042",
    "run_id": "run_20210121_06",
    "status": "candidate|pass|downgrade|watchlist|drop",
    "metric": "checkout_to_purchase_rate", // canonical (registry-validated)
```

```

"dimension_slice": {"channel_group": "Paid Search", "device_category": "mobile"},
"t0": {"start": "2020-12-01", "end": "2020-12-20"},
"t1": {"start": "2020-12-21", "end": "2021-01-10"},
"observed": {"value_t0": 0.061, "value_t1": 0.034, "delta": -0.027, "unit": "rate"},
"decomposition": {"mix_effect": -0.001, "rate_effect": -0.025, "interaction": -0.001,
                  "delta_R": -0.027, "dominant": "rate"},
"significance": {"test": "two_proportion_z", "p_value": 0.004, "ci_low": -0.034,
                 "ci_high": -0.020, "significant": true, "n_t0": 4120, "n_t1": 3980},
"dollar_impact": {"revenue_at_risk_usd": 42150.0, "basis": "(conv_t0-conv_t1)*sessions_t1*aov"},
"evidence": {"query_ids": ["q_17", "q_22"], "reconciled": true, "reconcile_drift": 0.001},
"critic": {"verdict": "pass", "rounds": 1, "refutations_tried": ["mix_confound", "seasonality", "sam
"recommendation": {"hypothesis": "simplify mobile payment step", "experiment_card_id": "H-2021-004
"priors": {"recalled": ["dx_20210115_07"], "suppressed": false}
}

```

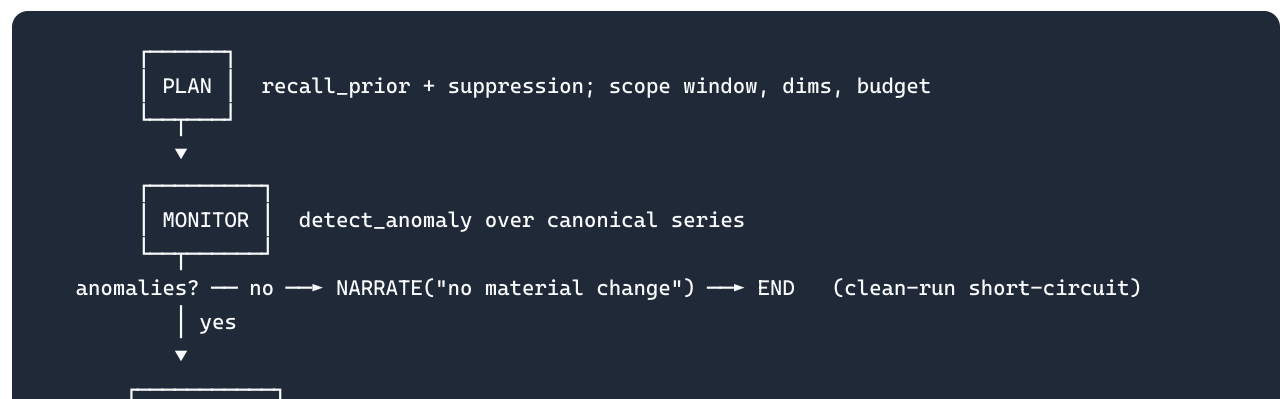
The runner validates each node's output against `output_schema`; a malformed envelope is a node failure (retry, then prune).

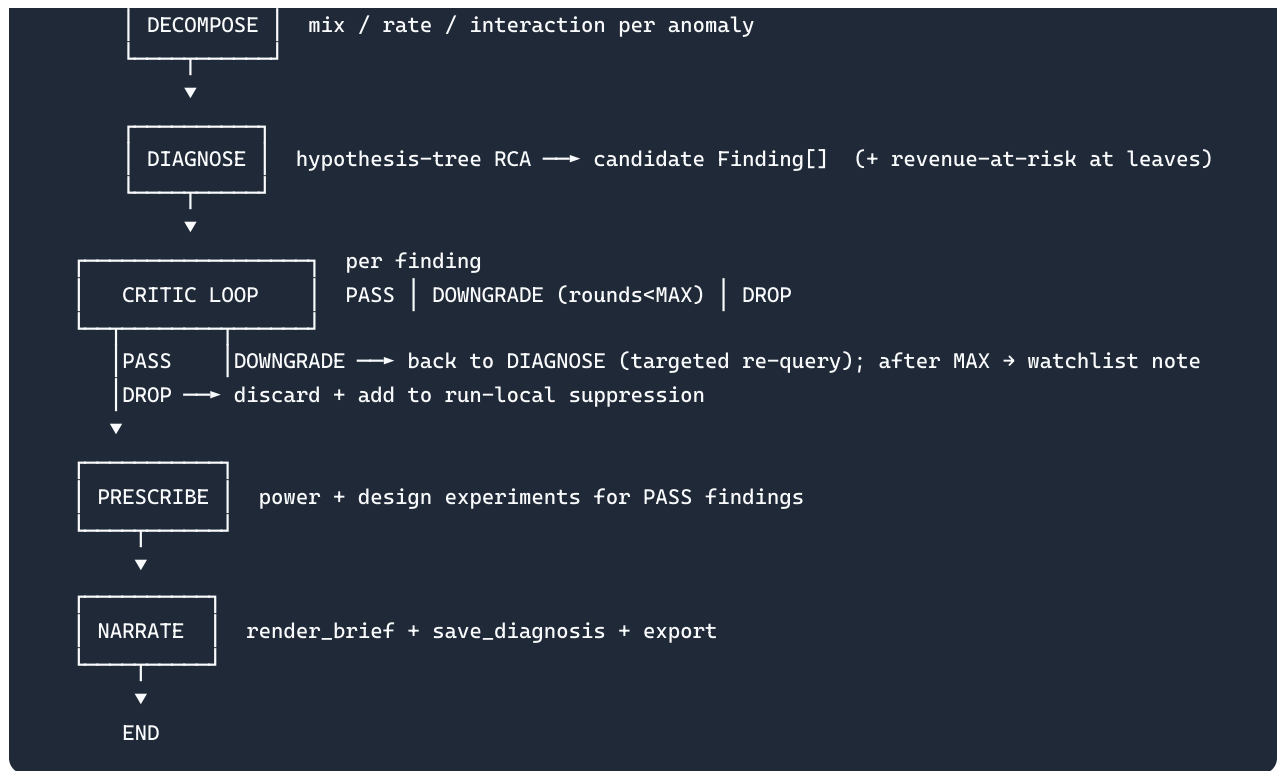
4.4 Context management

- Each node receives a **compacted** context: the run plan, the canonical metric/dimension catalog (names only), the upstream stage's `Finding[]` (JSON), and any `recall_prior` priors.
- Never** row-level dumps: `run_query` results are reduced to aggregates / decomposition inputs before entering the LLM window. Token usage stays flat as the hypothesis tree widens.
- The Diagnose tree's intermediate nodes are summarized to `(slice, rate_effect, p, n)` tuples, not full result sets.

5. Control flow — the FSM

The Orchestrator drives this machine. Worker output never ships directly; it must survive the Critic.





5.1 Transition table

From	Condition	To
PLAN	scope + priors loaded	MONITOR
MONITOR	≥ 1 anomaly	DECOMPOSE
MONITOR	0 anomalies	NARRATE (no-finding brief) → END
DECOMPOSE	always	DIAGNOSE
DIAGNOSE	candidate findings produced	CRITIC (per finding)
CRITIC	verdict = PASS	finding → PASS set
CRITIC	verdict = DOWNGRADE $\wedge$ rounds < MAX_REFUTE_ROUNDS	DIAGNOSE (targeted re-query)
CRITIC	verdict = DOWNGRADE $\wedge$ rounds $\geq$ MAX	finding → watchlist note
CRITIC	verdict = DROP	discard + run-local suppression
(all findings resolved)	PASS set ≥ 0	PRESCRIBE

From	Condition	To
PRESCRIBE	always	NARRATE
NARRATE	brief rendered + persisted	END
any stage	hard failure / budget exhausted / integrity violation	ABORTED (audit row; no partial brief)

6. Per-agent specifications

Each spec: responsibility · inputs · outputs · key logic · system-prompt sketch. (Tools per §2.1; models per §3.)

6.1 Orchestrator (Opus)

- **In:** run config (window, budget). **Out:** run plan; final brief pointer; `run_state` row.
- **Logic:** opens the `run_state` row (PLAN); calls `recall_prior` to load priors + suppression; sets scope (which metrics/dims, which window $t_0 \rightarrow t_1$) and the per-run byte/time budget; drives the FSM; routes each candidate to the Critic; finalizes (or ABORTs with an audit row).
- **System prompt (sketch):** *"You plan and supervise an autonomous funnel-diagnosis run. You do not analyze data yourself. Decide scope and budget, sequence the stages, route every finding through the Critic, and stop when the budget is spent or the brief is ready. Obey G1–G5. Output the run plan and state transitions as JSON."*

6.2 Monitor (Sonnet)

- **In:** scope. **Out:** anomaly list `[{metric, dim, t0, t1, score, direction}]`.
- **Logic:** for each canonical metric series, `build_query` → `dry_run` → `run_query` to fetch the daily series, then `detect_anomaly` (and `forecast` for expected-vs-actual). Flags points beyond `ANOMALY_SCORE_THRESHOLD`, seasonality-aware. Emits zero anomalies → clean-run short-circuit.
- **Prompt (sketch):** *"Detect material movements in the canonical metric series. Get series only via `build_query`; score only via `stats.detect_anomaly/forecast` — never eyeball. Return a ranked anomaly list as JSON. Do not diagnose."*

6.3 Decompose (Sonnet)

- **In:** anomaly list. **Out:** decomposition per anomaly.

- **Logic:** for each anomaly, `build_query` the segment table `[{seg, w, r}]` at `t0` and `t1` across the candidate dimension, then `stats.decompose_change → {mix, rate, interaction, by_segment}`. Labels each anomaly's `dominant` effect.
- **Prompt (sketch):** "For each anomaly, compute the mix/rate/interaction split via `stats.decompose_change` over the relevant dimension. Return the decomposition. Never compute the split yourself."

6.4 Diagnose (Opus) — the hypothesis tree

- **In:** decompositions. **Out:** candidate `Finding[]` with evidence + revenue-at-risk.
- **Algorithm — best-first search over the dimensional space:**
 - **Root** = the moved top-level metric (e.g. `session_conversion_rate -2.1pp`).
 - **Node** = `(metric, dimension-slice, decomposition verdict)`. **Expand** along the next canonical dimension, ordered by **rate-effect magnitude** from `decompose_change` — always drill the slice with the largest *rate* change (genuine behavior change) before chasing *mix* effects (composition artifacts / Simpson's paradox).
 - **Prune** a branch when: `rate-effect < MIN_RATE_EFFECT`, OR `significance_test p > SIGNIFICANCE_ALPHA`, OR `slice sessions < MIN_SEGMENT_SESSIONS`.
 - **Bounds:** `MAX_BRANCHING` slices/node, `MAX_DEPTH` dimensions — keeps cost and <5 min/run in budget.
 - **Leaf promotion → candidate Finding** when the node: is statistically significant, isolates a single dimension's rate change, survives `reconcile` (G4, ≤0.5% drift), and carries a non-trivial dollar impact.
 - **QUANTIFY at the leaf:** `revenue_at_risk = (r_counterfactual - r_observed) × affected_sessions × downstream_value`. For a `session_conversion_rate` leaf: `(conv_t0 - conv_t1) × sessions_t1 × aov`. `r_counterfactual` is the `t0` value or the `forecast` baseline. Computed entirely from governed metrics.
- **Prompt (sketch):** "Find the root cause by best-first search: drill the slice with the largest rate-effect first; verify every branch with `build_query→dry_run→run_query` and `significance_test`; prune insignificant/low-effect/low-sample branches; promote a leaf only if significant, single-dimension, reconciled, and dollar-material. Compute revenue-at-risk from governed metrics. Return candidate Findings as JSON. Never write SQL or compute stats yourself."

6.5 Critic (Opus) — adversarial verifier

- **In:** a candidate `Finding` + its evidence. **Out:** verdict + reasons.
- **Refutation battery (tries to DROP the finding):**
 1. **Mix-shift confound** — re-run `decompose_change`; if the move is actually `dominant = mix`, the finding's rate story is refuted.

2. **Insufficient sample** — re- `significance_test` on a holdout/adjacent slice; small `n` or `p > α` → DOWNGRADE.
 3. **Seasonality** — `recall_prior` + the `seasonality_calendar`; if the change is within `expected_mag` of a known calendar entry overlapping `[t0,t1]` → DROP/DOWNGRADE as expected.
 4. **Data quality** — confound probe via `run_query` (NULL spikes, `transaction_id` duplication, late shard); if explained by data quality, DROP.
- **Verdicts: PASS** (survives all) → Prescribe. **DOWNGRADE** (plausible, weak evidence) → back to Diagnose, bounded by `MAX_REFUTE_ROUNDS`; after the cap → *watchlist note*, not a confident finding. **DROP** (refuted) → discard + run-local suppression so siblings don't re-raise it.
 - **Prompt (sketch):** *"Your job is to REFUTE this finding, not confirm it. Attack it on four axes: is it a mix-shift not a rate change? is the sample too small? is it known seasonality/launch (check recall_prior)? is it a data-quality artifact? Default to skepticism. Return PASS only if every refutation fails. Output verdict + the refutations you tried + reasons."*

6.6 Prescribe (Sonnet)

- **In:** PASS `Finding[]`. **Out:** hypothesis cards / prioritized backlog.
- **Logic:** for each PASS finding, `power_analysis(baseline, mde)` → `n`; `runtime_estimate(n, traffic)` → `days`; `design_experiment(hypothesis, metric)` → test card (primary + guardrail metrics, MDE, `n`, runtime). Prioritize by ICE (Impact = `revenue_at_risk × expected_lift`; Confidence from evidence strength + priors; Effort estimate). Only governed metrics may be targeted.
- **Prompt (sketch):** *"Turn each confirmed finding into a powered, prioritized experiment card via experiment-mcp. Rank by ICE using the finding's revenue-at-risk. Target only canonical metrics. Output cards as JSON."*

6.7 Narrator (Sonnet)

- **In:** PASS findings + backlog. **Out:** Decision Brief (md/html); persisted diagnosis.
- **Logic:** `render_brief(findings)` → exec brief where each finding shows decomposition + significance + dollar revenue-at-risk + recommended action; `save_diagnosis` (writes memory — A7.4, full loop); `export` (pdf/slack). **Faithfulness rule:** every numeric claim must trace to a tool-output hash; no number the Narrator invented.
- **Prompt (sketch):** *"Compose a one-page executive Decision Brief from the PASS findings, ranked by dollars at risk. Every number must come from the finding's evidence — never introduce a figure. Then save_diagnosis and export. Output the brief."*
- **Minimal-loop (M7) vs full (M9):** at M7 the Narrator only calls `render_brief` (report-mcp core, A6.5); `save_diagnosis` is added at M8 when memory tools (A7.4) exist.

7. Grounding rules — double enforcement (G1–G5)

Rule	Structural enforcement	Behavioral enforcement
G1 never raw SQL	no agent holds a raw-SQL tool; only <code>build_query</code>	system prompts forbid it
G2 never compute stats	only <code>stats-mcp</code> / <code>experiment-mcp</code> produce numbers	prompts: "numbers are tool outputs verbatim"
G3 <code>dry_run</code> before <code>run_query</code>	wrapper asserts <code>dry_run_seen(sql)</code>	—
G4 reconcile	leaf-promotion requires <code>reconcile ≤ 0.5% drift</code>	Critic re-checks
G5 canonical names only	<code>semantic-mcp</code> raises on unknown names	prompts give the canonical catalog; glossary resolves synonyms before reaching <code>semantic-mcp</code>

Structural beats behavioral: even a misbehaving prompt cannot exceed the allow-list.

8. Memory integration (§22 coupling — the LEARN step)

All agent memory I/O goes through `report-mcp` (`recall_prior` / `save_diagnosis` , A7.4); the LLM never writes memory directly.

- **Orchestrator @ PLAN** — `recall_prior(metric, segment)` loads: similar prior findings (vector ANN, decayed `exp(-age/60d)`), the suppression list, and `action_tracking` outcomes.
- **Diagnose** — down-weights a root cause whose prescribed fix already shipped and won (`action_tracking.outcome = win`).
- **Critic** — uses `seasonality_calendar` / `launch_calendar` priors to refute "expected" moves; honors the suppression list (auto-DROP a suppressed (`metric, slice, root_cause`)).
- **Narrator** — `save_diagnosis` persists every PASS (and DROP, for audit) with an embedding; Prescribe writes the experiment as `action_tracking(proposed)` .
- **Loop closure:** a later run sees "seen before / already fixed / known-seasonal / was-a-bad-diagnosis" and behaves accordingly. (Full schemas: Bible §22.)

9. Configuration constants (the tunables)

Central config (`agents/config.py`); defaults chosen for the ~3-month dataset window and the <5 min/run budget.

Constant	Default	Meaning
<code>MAX_REFUTE_ROUNDS</code>	2	Critic→Diagnose re-query rounds before watchlist
<code>MAX_BRANCHING</code>	4	slices expanded per hypothesis-tree node
<code>MAX_DEPTH</code>	3	dimensions deep the tree may go
<code>MIN_RATE_EFFECT</code>	0.3 pp	prune branches below this rate-effect
<code>MIN_SEGMENT_SESSIONS</code>	500	minimum sample to consider a slice
<code>SIGNIFICANCE_ALPHA</code>	0.05	significance threshold
<code>ANOMALY_METHOD</code>	<code>"stl"</code>	default detector (EWMA/IQR fallbacks)
<code>ANOMALY_SCORE_THRESHOLD</code>	3.0	flag points beyond this score
<code>PRIOR_HALF_LIFE_DAYS</code>	60	memory recall decay
<code>BYTE_BUDGET</code>	5 GiB	per-run cap (shared w/ warehouse-mcp)
<code>TIME_BUDGET_MIN</code>	5	wall-clock target per run
<code>MODELS</code>	Opus: {Orchestrator, Diagnose, Critic}; Sonnet: {Monitor, Decompose, Prescribe, Narrator}	model assignment
<code>TEMPERATURE</code>	0.0	low; determinism where the model allows

10. End-to-end run (the A9.1 runner)

```
Scheduler —fire→ Orchestrator
Orchestrator → recall_prior( ... )           # priors + suppression; open run_state(PLAN)
Orchestrator → list_tables / list_dimensions  # scope
Orchestrator → Monitor("window t0..t1, budget B")
    Monitor → build_query(session_conversion_rate,[day],window) → dry_run → run_query
    Monitor → detect_anomaly(series) → [conv_rate -2.1pp]
Orchestrator → Decompose(anomaly list)
    Decompose → decompose_change(conv_rate, device_category, t0, t1) → {mix:+0.3, rate:-1.8, int:-0
Orchestrator → Diagnose(decompositions)
    Diagnose → (tree) build_query → run_query ; decompose_change ; significance_test ; reconcile
    Diagnose → candidate F1 (/sale, Paid Search, mobile; p=0.01; $42k risk)
Orchestrator → Critic(F1 + evidence)
    Critic → recall_prior ; significance_test(holdout) ; run_query(confound probe)
    Critic → VERDICT = PASS
Orchestrator → Prescribe(F1)
    Prescribe → power_analysis → runtime_estimate(→9d) → design_experiment → Exp-001
Orchestrator → Narrator(PASS findings + backlog)
    Narrator → render_brief → save_diagnosis → export(pdf/slack)
Orchestrator → close run_state(END); audit row written
```

A **clean run** (no anomaly) short-circuits after Monitor: Narrator emits a "no material change" brief and the run still writes a `run_state` row, so *absence of finding* is itself recorded.

11. Reliability, determinism, reproducibility

- **Retries.** Transient `warehouse` /BigQuery errors → exponential backoff (≤ 3). A `dry_run` budget rejection (G3) is **not** retried blindly — the agent must narrow window/dims first. A `semantic` unknown-name error (G5) is a hard stop for that branch; the agent re-plans against `list_dimensions()`.
- **Failure isolation.** A single failed Diagnose branch is logged and pruned, not fatal. Total failure of Monitor/Decompose → the Orchestrator ABORTs with an audit record; **no partial brief ships**.
- **Budget.** The wrapper tracks cumulative `bytes_scanned` against `BYTE_BUDGET` mid-run; nearing the cap, the Orchestrator tightens scope. Token cost is bounded by the Opus/Sonnet split + compaction (§4.4).
- **Determinism (honest statement).** Stats and SQL are deterministic (seeded; governed). LLM nodes are run at `temperature 0` but are **not** byte-deterministic — so the agent layer is graded **statistically by the offline eval harness** (root-cause accuracy $\geq 85\%$), not asserted byte-identical. The full `audit_log` makes every run reconstructable regardless.

12. Testing & evaluation hooks

- **Node unit tests:** each agent tested with **recorded tool fixtures** (canned MCP responses) → assert the returned `Finding[]` envelope (schema + key fields). No live BigQuery needed.
 - **FSM tests:** drive `transition()` with synthetic envelopes → assert correct routing (clean-run short-circuit; DOWNGRADE→re-query bounded by MAX; DROP→suppression; ABORT on failure).
 - **Critic tests:** feed known-confounded findings (a pure mix-shift labeled as rate) → assert DROP/DOWNGRADE; feed a clean finding → assert PASS.
 - **Integration:** the **eval harness (A10)** points this FSM at `helios_eval.fct_daily_funnel_perturbed` and grades root-cause accuracy / decomposition error / hallucination / faithfulness against injected labels (Bible §20). This is the agent layer's real grade.
 - **Grounding test:** scan the run's `audit_log` — every `sql_text` must originate from `semantic.build_query` (0 hand-authored SQL) and every brief number must map to a tool output (faithfulness).
-

13. Build order within the agent layer (M7 → M9)

1. **A8.0 framework** — `AgentSpec`, the tool-call wrapper (allow-list + audit + budget + retries), the `Finding` schema/validator, context compaction. (Unblocks all agents.)
2. **M7 minimal loop (L1):** `A8.1 Monitor` + `A8.6 Narrator` (render_brief only) + the `A9.1` runner with a trivial PLAN→MONITOR→NARRATE path. **Gate:** one anomaly → brief in <5 min, 0 hallucinated columns.
3. **M9 full loop (L2):** add `A8.2 Decompose`, `A8.3 Diagnose` (hypothesis tree), `A8.4 Critic` (refutation loop), `A8.5 Prescribe`, `A8.7 Orchestrator`, `A9.3` audit wiring; Narrator gains `save_diagnosis` (needs memory tools A7.4 from M8). **Gate:** a real run produces PASS findings each carrying significance + dollar impact + a recommended experiment; eval harness (A10) measures ≥85% root-cause accuracy vs ≤45% baseline.

Exit gate (L2): the FSM, pointed at perturbed eval data, rediscovers injected root causes at ≥85% top-1, with every shipped finding decomposed, significance-tested, dollar-quantified, Critic-approved, and fully traceable in the audit log.